Drone system : navigation system

Par Moustafa KASSIR et Gaëtan BOUCHY

Sommaire

- Introduction
- Diagramme Use-Case
- Diagramme de composition
- Machine à états
- Diagramme de profils
- Diagramme d’activité
- Conclusion

Introduction



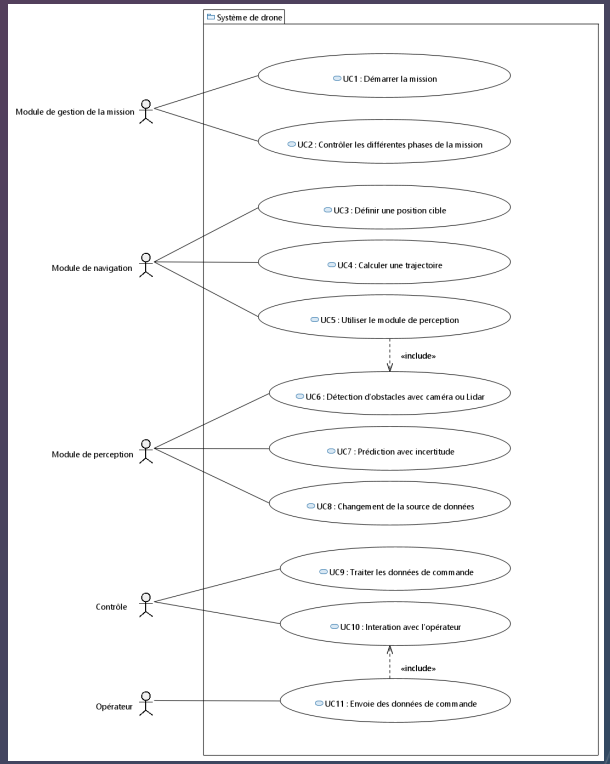



- Drone livreur de colis
- Destination A à une destination B
- Capteurs de guidage
- Contrôle manuel

Diagramme Use-Case

Le diagramme Use-Case modélise les interactions entre un système et ses utilisateurs, en se concentrant sur les fonctionnalités du système plutôt que sur sa mise en œuvre technique.

Diagramme Use-Case



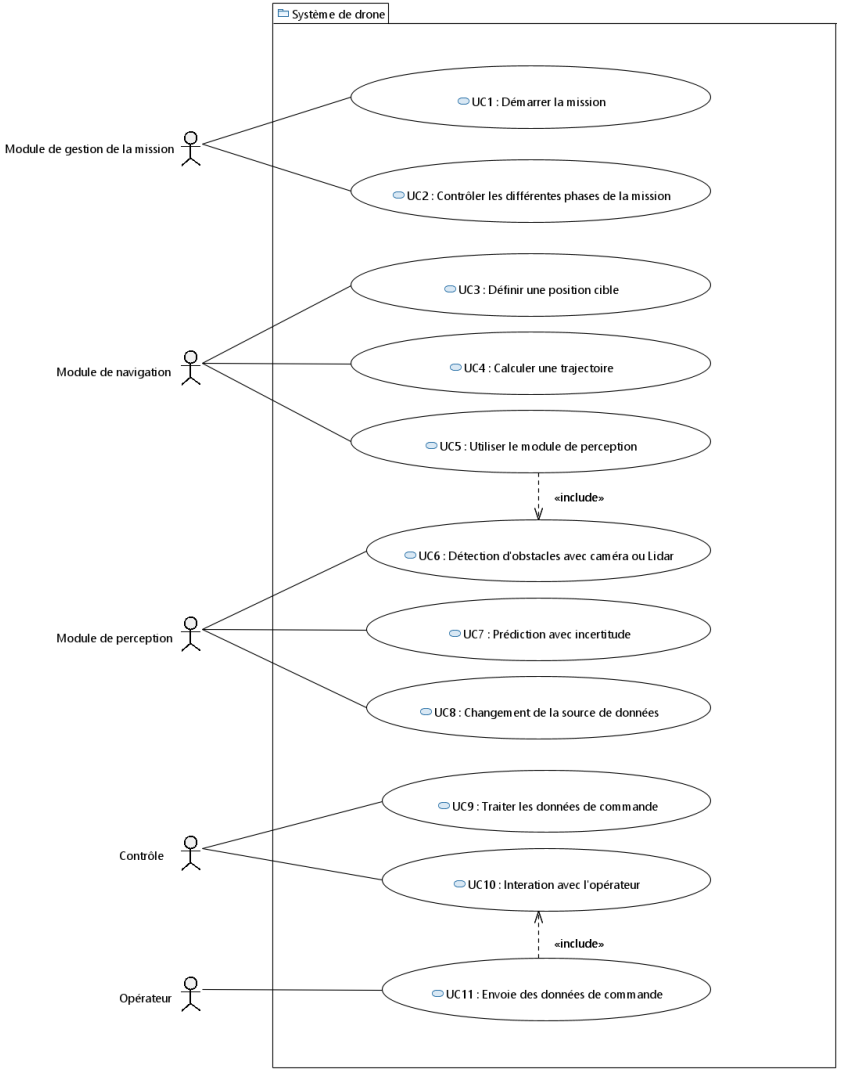
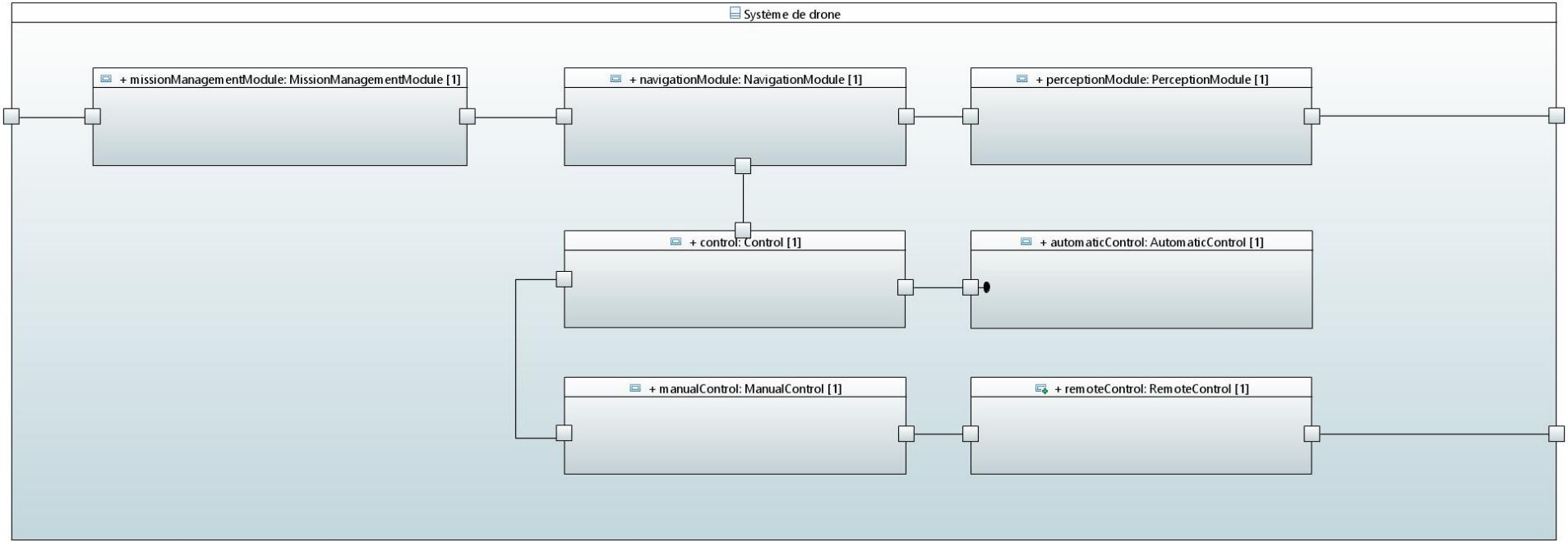

Système de drone
Module de gestion de la mission
UC1 : Démarrer la mission
UC2 : Contrôler les différentes phases de la mission
Module de navigation
UC3 : Définir une position cible
UC4 : Calculer une trajectoire
UC5 : Utiliser le module de perception
«include»
Module de perception
UC6 : Détection d'obstacles avec caméra ou Lidar
UC7 : Prédiction avec incertitude
UC8 : Changement de la source de données
Contrôle
UC9 : Traiter les données de commande
UC10 : Interation avec l'opérateur
«include»
Opérateur
UC11 : Envoie des données de commande

Diagramme de composition

Le diagramme de composition est utilisé pour montrer la composition d’un système à partir de ses parties qui le constitue, en illustrant comment celles-ci sont liées et interagissent entre elles pour produire le comportement global du système.

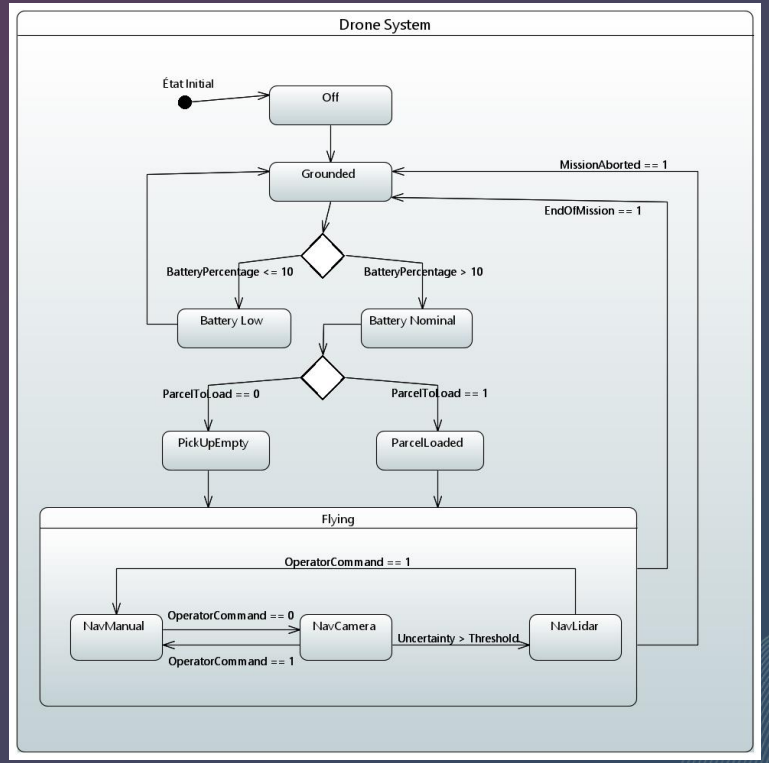
Diagramme de composition

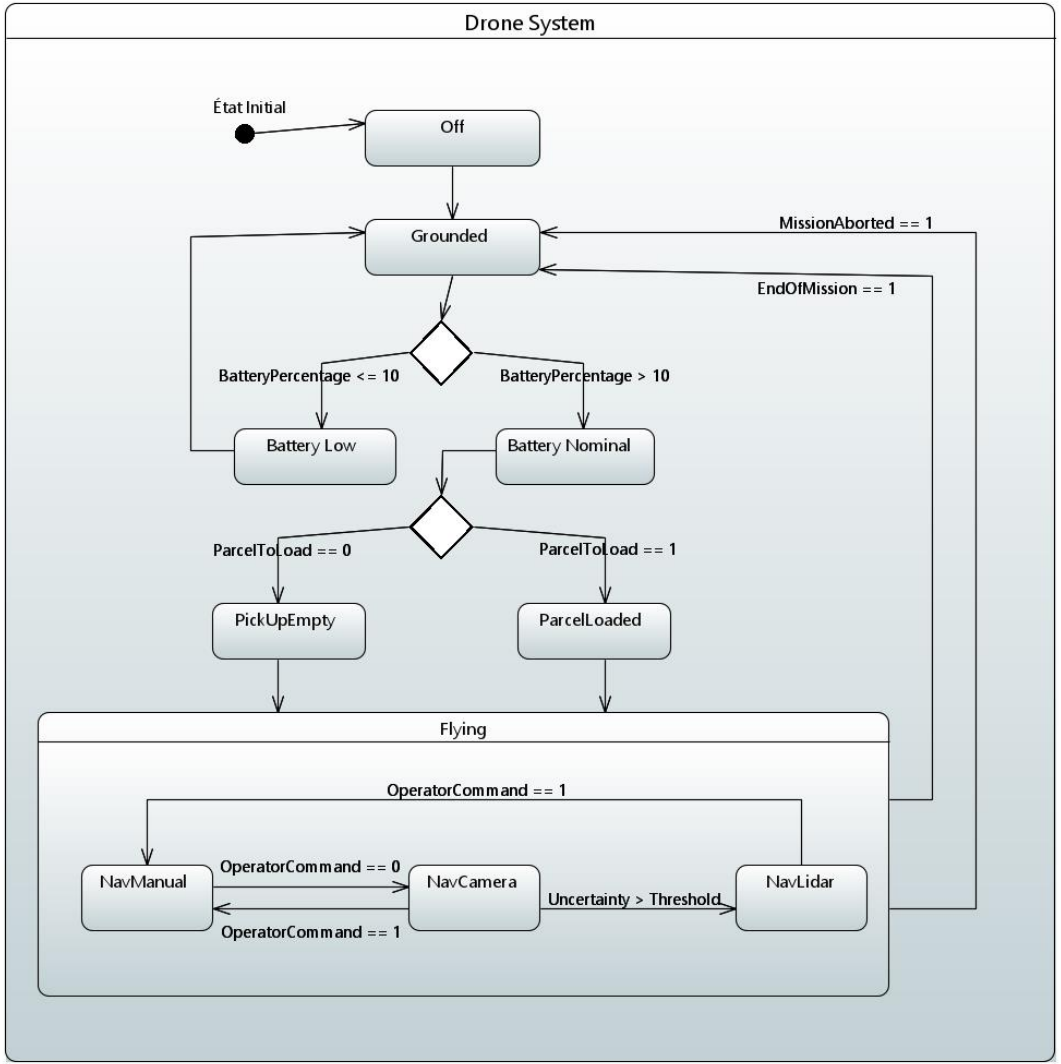


Machines à états

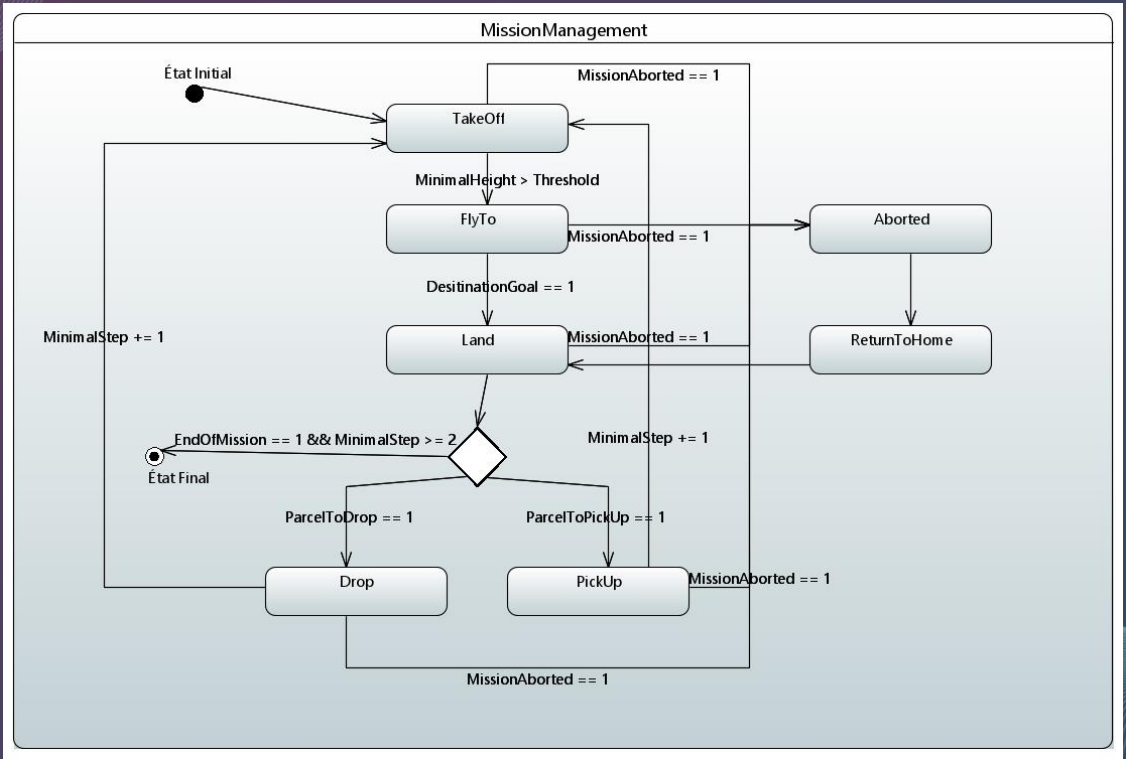
Les machines à états sont utilisés pour modéliser le comportement des objets dans des systèmes. Une machine à états décrit les différents états qu'un objet peut prendre et comment il passe d'un état à un autre en réponse aux événements qui se produisent.

Machines à états (1/2)




Drone System
État Initial
Off
Grounded
MissionAborted == 1
EndOfMission == 1
BatteryPercentage <= 10
BatteryPercentage > 10
Battery Low
Battery Nominal
ParcelToLoad == 0
ParcelToLoad == 1
PickUpEmpty
ParcelLoaded
Flying
OperatorCommand == 1
NavManual
OperatorCommand == 0
NavCamera
Uncertainty > Threshold
NavLidar
OperatorCommand == 1

Machines à états (2/2)



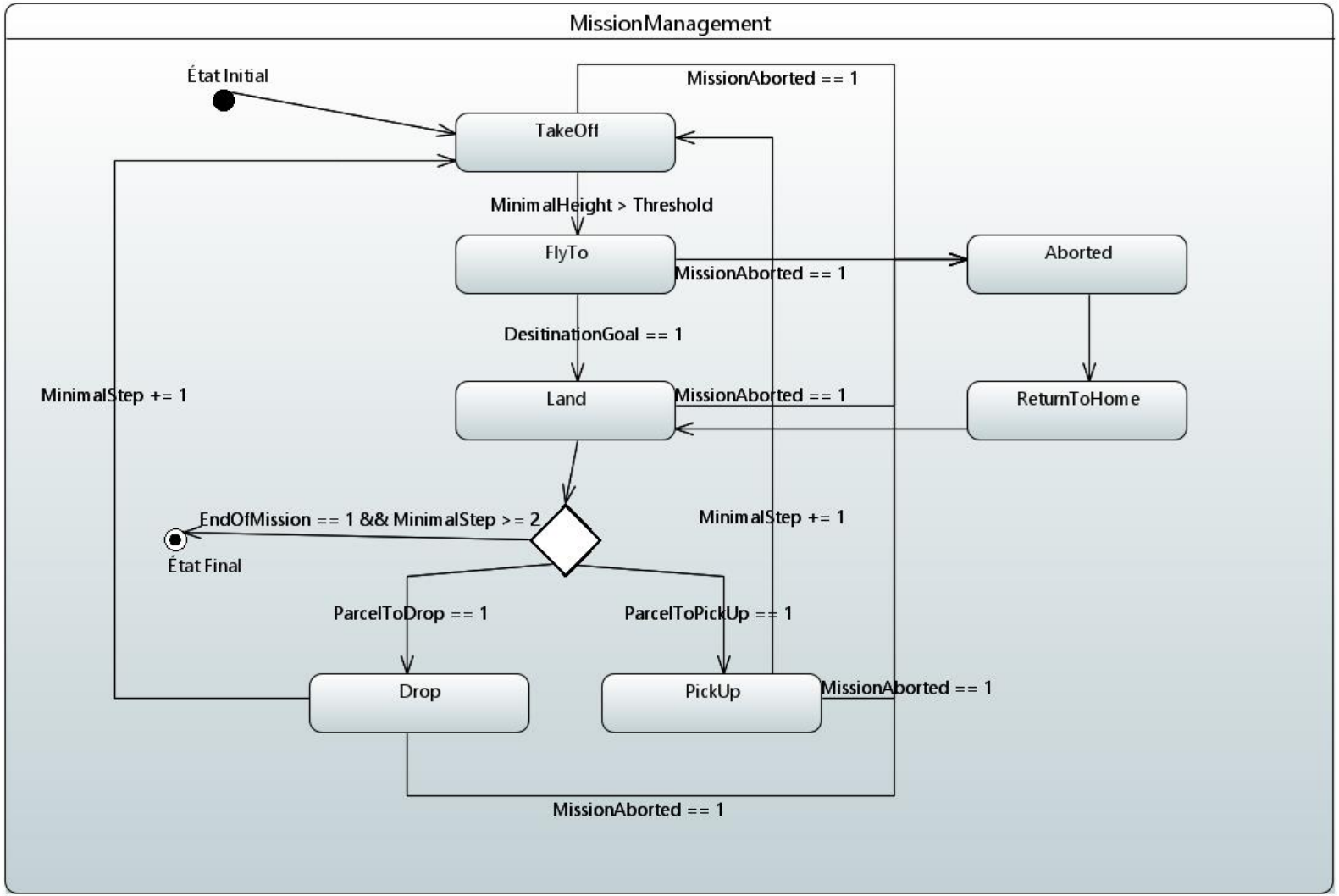
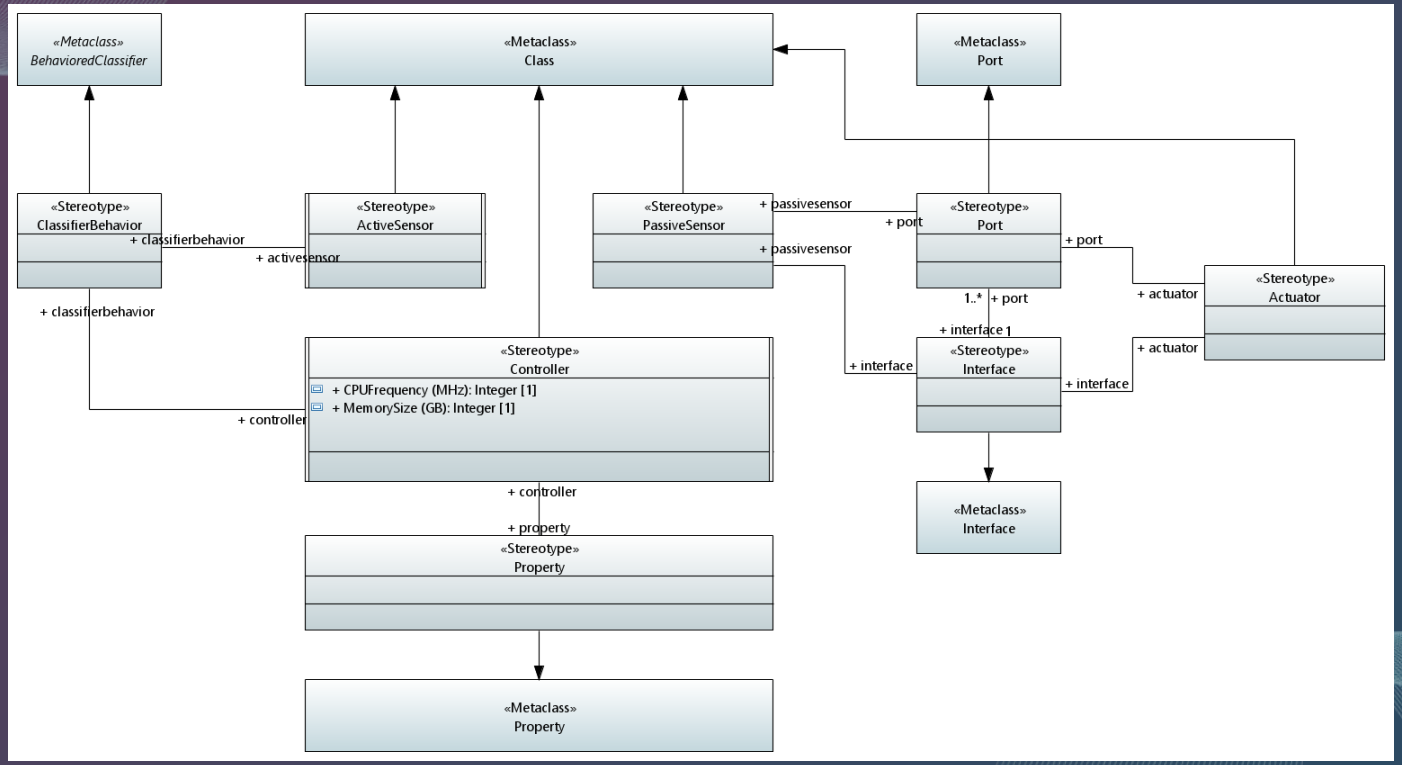
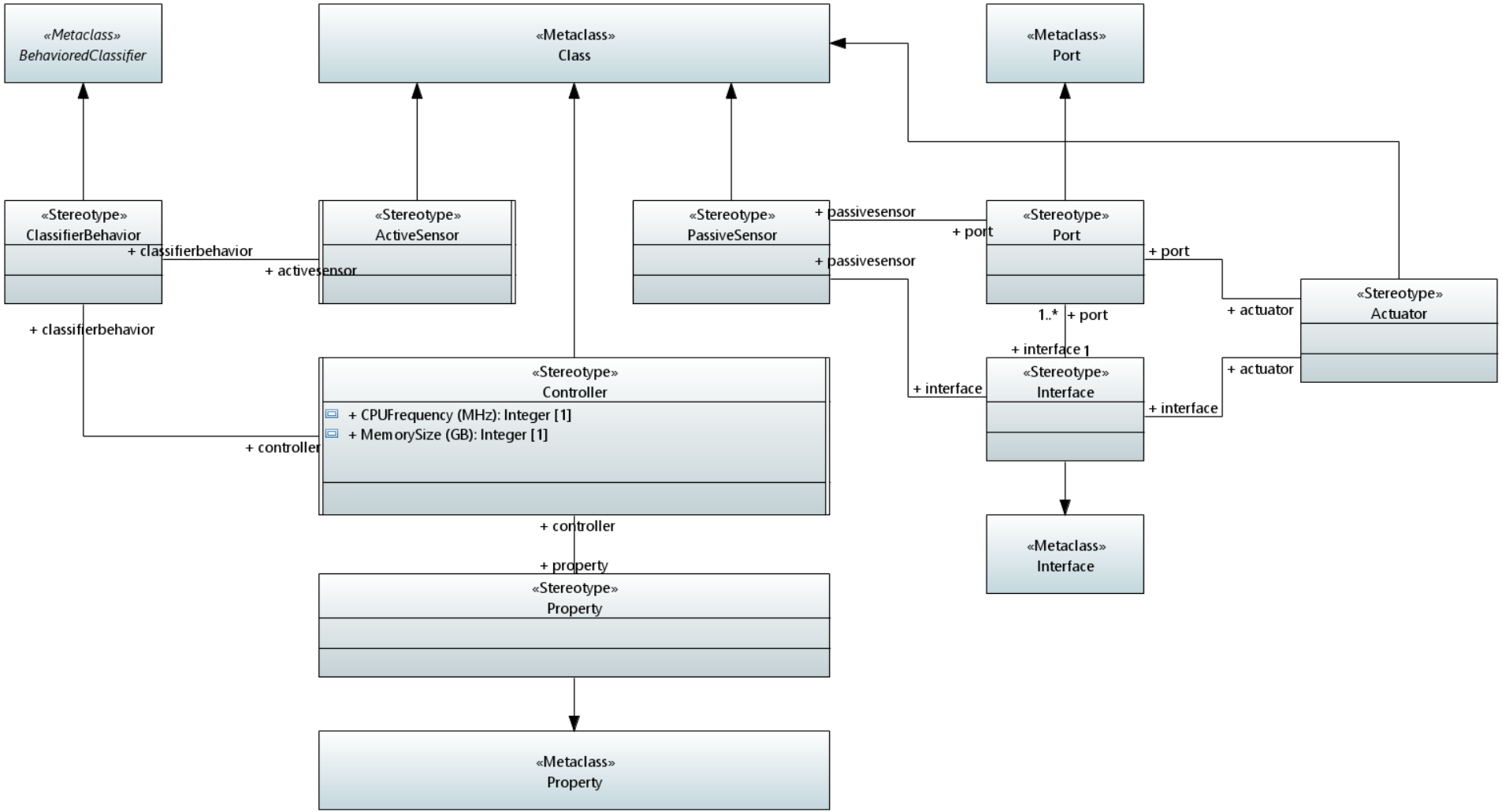

MissionManagement
État Initial
MissionAborted == 1
TakeOff
MinimalHeight > Threshold
FlyTo
Aborted
MissionAborted == 1
DesitinationGoal == 1
MinimalStep += 1
Land
MissionAborted == 1
ReturnToHome
EndOfMission == 1 && MinimalStep >= 2
MinimalStep += 1
État Final
ParcelToDrop == 1
ParcelToPickUp == 1
Drop
PickUp
MissionAborted == 1
MissionAborted == 1

Diagramme de profils

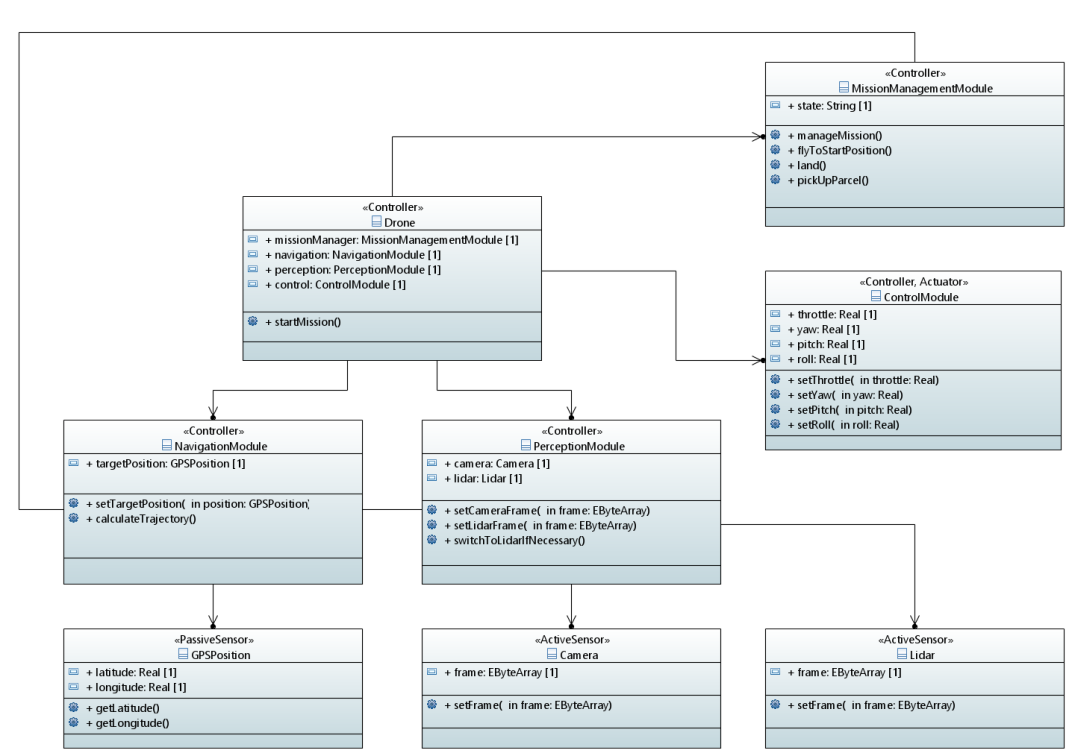
Les diagrammes de profils sont utilisés pour étendre le langage UML et personnaliser la notation et la sémantique. Un profil UML est une extension d'UML qui peut être utilisée pour ajouter des stéréotypes, des contraintes, des propriétés ou des méthodes à un modèle UML de base.

Diagramme de profils




«Metaclass»
BehavioredClassifier
«Metaclass»
Class
«Metaclass»
Port
«Stereotype»
ClassifierBehavior
+ classifierbehavior
+ classifierbehavior
«Stereotype»
ActiveSensor
+ activesensor
«Stereotype»
PassiveSensor
+ passivesensor
+ passivesensor
«Stereotype»
Port
+ port
+ port
1..*
+ port
«Stereotype»
Actuator
+ actuator
+ actuator
+ interface 1
«Stereotype»
Interface
+ interface
+ interface
«Stereotype»
Controller
+ CPUFrequency (MHz): Integer [1]
+ MemorySize (GB): Integer [1]
+ controller
+ controller
+ property
«Metaclass»
Interface
«Stereotype»
Property
«Metaclass»
Property

Application du diagramme de profils



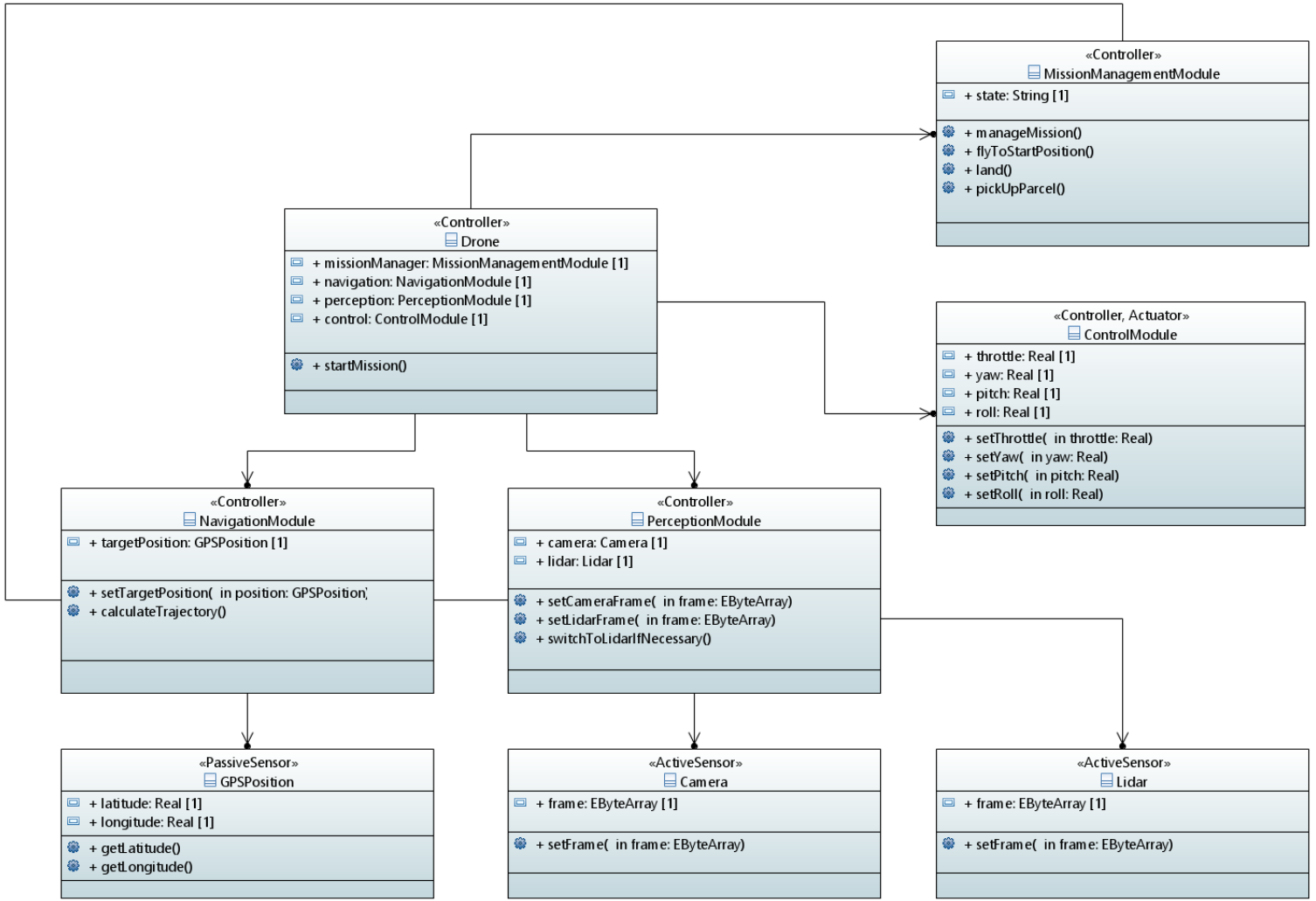
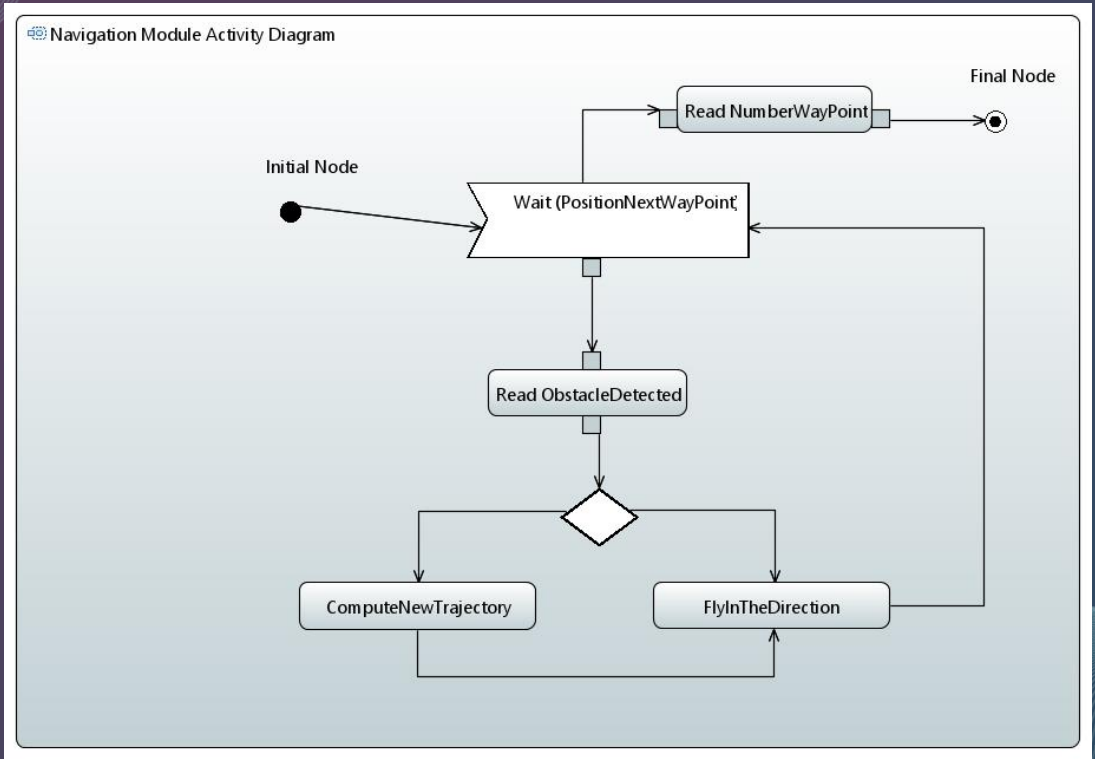
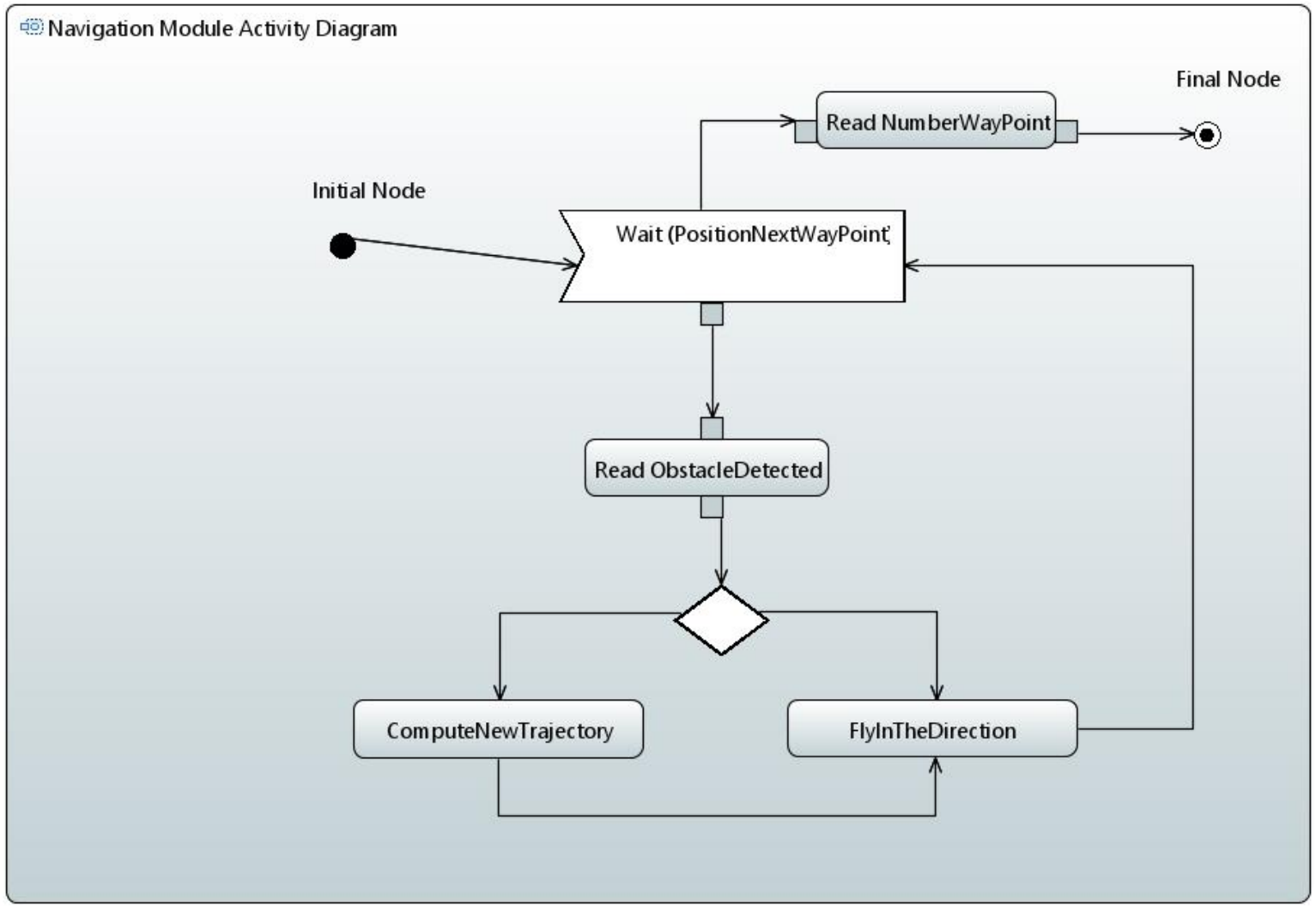

«Controller»
MissionManagementModule
+ state: String [1]
+ manageMission()
+ flyToStartPosition()
+ land()
+ pickUpParcel()
«Controller»
Drone
+ missionManager: MissionManagementModule [1]
+ navigation: NavigationModule [1]
+ perception: PerceptionModule [1]
+ control: ControlModule [1]
+ startMission()
«Controller, Actuator»
ControlModule
+ throttle: Real [1]
+ yaw: Real [1]
+ pitch: Real [1]
+ roll: Real [1]
+ setThrottle(in throttle: Real)
+ setYaw(in yaw: Real)
+ setPitch(in pitch: Real)
+ setRoll(in roll: Real)
«Controller»
NavigationModule
+ targetPosition: GPSPosition [1]
+ setTargetPosition(in position: GPSPosition)
+ calculateTrajectory()
«Controller»
PerceptionModule
+ camera: Camera [1]
+ lidar: Lidar [1]
+ setCameraFrame(in frame: EByteArray)
+ setLidarFrame(in frame: EByteArray)
+ switchToLidarIfNecessary()
«PassiveSensor»
GPSPosition
+ latitude: Real [1]
+ longitude: Real [1]
+ getLatitude()
+ getLongitude()
«ActiveSensor»
Camera
+ frame: EByteArray [1]
+ setFrame(in frame: EByteArray)
«ActiveSensor»
Lidar
+ frame: EByteArray [1]
+ setFrame(in frame: EByteArray)

Diagramme d’activité

Le diagramme d'activité est utilisé pour modéliser le comportement d'un processus ou d’un flux dans un système. Il permet de représenter les activités, les actions, les étapes de décision, les synchronisations, les transitions... qui composent un processus

Diagramme d’activité




Navigation Module Activity Diagram
Final Node
Read NumberWayPoint
Initial Node
Wait (PositionNextWayPoint)
Read ObstacleDetected
ComputeNewTrajectory
FlyInTheDirection

Conclusion






- Représentation graphique des différents aspects d'un système (fonctionnalités, architecture, comportements, ...)
- Clarification des exigences, des relations et des interactions en offrant une représentation visuelle

Merci de votre attention